HoloCubic「AI 状态灵动岛」零基础使用指南

> 适用：HoloCubic 透明小电视 + HoloCubic_AIO 固件 v2.1.13 + 新增的 `AIStatus` APP；电脑为 macOS，使用 Wi-Fi。
>
> 效果：Claude Code / Codex / Grok 工作时，小电视实时显示 **谁、在干什么、多久了**，RGB 灯同步变色。
>
> 标注说明：**【已核对代码】** 表示根据仓库源码确认；**【未实机验证】** 表示还没有在真实硬件/Mac 上跑过，请以实际为准。

---

0. 整体原理（一分钟看懂）

```
AI 工具 (Claude/Codex/Grok) ──hook 触发──► ~/.holocube/holocube_notify.sh
          ──curl POST JSON──► http://小电视IP:8080/status ──► 屏幕 + RGB 灯
```

- 小电视和 Mac 必须在**同一个 Wi-Fi（同一网段）**。
- 只有在小电视上**进入 AIStatus APP 后**，8080 端口才会监听。

---

1. 准备

1.1 硬件

物品	说明
HoloCubic 小电视	已组装好的 ESP32 版（Naive / Metal 均可）。
USB 数据线	**必须能传数据**。很多随手拿的线只能充电，插上后电脑完全没反应。
microSD 卡（可选）	AIStatus 没有 SD 卡也能用；**要显示中文需要 SD 卡**放字体（见 5.2）。相册、视频、天气图标等其他 APP 才需要。官方 Q&A 推荐爱国者 32G、闪迪 32G 普通卡。
2.4 GHz Wi-Fi	ESP32 **不支持 5 GHz**。双频合一的路由器建议单独开一个 2.4G 名称。
供电	电脑 USB 口或 5V/1A 以上充电头。供电不足时，连 Wi-Fi 那一刻容易重启。

1.2 Mac 软件

1. **VS Code**：https://code.visualstudio.com/ 下载后拖进「应用程序」。
2. **PlatformIO IDE 插件**：VS Code 左侧「扩展」(四个方块图标) → 搜 `PlatformIO IDE` → Install。装好后会自动下载工具链，第一次要几分钟到十几分钟，完成后重启 VS Code。
3. **Git**（可选）：终端输入 `git --version`。如果弹出安装「命令行开发者工具」的提示，点安装即可。

1.3 判断串口芯片、要不要装驱动

先**不插**小电视，打开「终端」(启动台搜 Terminal)，运行：

```
ls /dev/cu.*
```

再**插上**小电视，再运行一次，**多出来的那一项**就是小电视：

多出来的名字（示例）	芯片	驱动
`/dev/cu.SLAB_USBtoUART` 或 `/dev/cu.usbserial-0001`	CP210x（CP2102）	新版 macOS 通常免驱；不出现就装 Silicon Labs「CP210x VCP Mac」驱动
`/dev/cu.wchusbserial*` 或 `/dev/cu.usbserial-14xx`	CH340 / CH343	新版 macOS 通常免驱；不出现就装 WCH 官方 macOS 驱动
什么都没多	—	先换**数据线**、换 USB 口；再装驱动；装完到「系统设置 → 隐私与安全性」点「允许」，然后重启

> 官方 Q&A 提到原版主板用 **CP2102**【已核对文档】。你的板子具体用哪颗芯片，以上表判断为准。
>
> 也可以在「关于本机 → 系统报告 → USB」里看到 `CP210x` 或 `USB Serial (CH340)` 字样。

---

2. 获取代码并打补丁

二选一即可。

方式 A：直接用打包好的 zip（最简单）

1. 拿到 `holocubic_ai_status.zip`，双击解压。
2. 里面有 `AIO_Firmware_PIO/`（固件工程，已包含 AIStatus）和 `ai_status_bridge/`（电脑端脚本）。

方式 B：从官方仓库 clone，再打补丁

```
cd ~
git clone https://github.com/ClimbSnail/HoloCubic_AIO.git
cd HoloCubic_AIO
git apply --check ~/Downloads/ai_status.patch    # 先检查，不报错再继续
git apply ~/Downloads/ai_status.patch
```

补丁的改动如下：

- 新增 `AIO_Firmware_PIO/src/app/ai_status/`（5 个文件）。
- 修改 `src/app/app_conf.h`：加 `APP_AI_STATUS_USE 1`。
- 修改 `src/HoloCubic_AIO.cpp`：把 AIStatus 安装为菜单第一个 APP。
- 修正 LHLXW APP 中 3 处头文件大小写（`arduino.h`→`Arduino.h`、`EMOJI.h`→`emoji.h`）。
- 新增 `ai_status_bridge/`（电脑端脚本与说明）。

补丁基于 2026-10-14 的仓库主分支。官方之后有更新时，可能需要手动合并。

3. 刷固件

3.1 打开工程

VS Code → `File → Open Folder…` → 选择 **`AIO_Firmware_PIO`** 这一层文件夹（里面直接能看到 `platformio.ini`）。选错层级 PlatformIO 不会识别。

3.2 修改上传端口

打开 `platformio.ini`，找到：

```
upload_port = COM12
```

改成第 1.3 步看到的端口，并加一行串口监视端口：

```
upload_port = /dev/cu.usbserial-0001
monitor_port = /dev/cu.usbserial-0001
```

【已核对代码】工程原有配置如下：

- 默认环境 `HoloCubic_AIO_Releases`，另有调试环境 `HoloCubic_AIO_Debug`。
- `upload_speed = 921600`，`monitor_speed = 115200`。
- 分区表 `partitions-no-ota.csv`，即**没有 OTA**，只能用 USB 刷机。

3.3 编译 & 上传

VS Code 底部蓝色状态栏上的几个按钮：

- ✓：Build，编译。第一次会下载 ESP32 平台，较慢。
- →：Upload，编译并烧录。
- 🔌：Serial Monitor，查看串口日志。

不想点按钮，也可以用终端。终端里的 `pio` 命令需要在 PlatformIO 侧边栏 → `Miscellaneous → New Terminal` 里打开才可用：

```
pio run -e HoloCubic_AIO_Releases               # 编译
pio run -e HoloCubic_AIO_Releases -t upload     # 上传
pio device monitor -b 115200                    # 串口日志，Ctrl+C 退出
```

本补丁在 Linux 上实测编译通过：Flash 87.6%（3,501,384 / 3,997,696 字节），RAM 36.4%（静态）【已编译验证；Mac 上未实机验证】。

3.4 上传卡住 / 进不了下载模式

- 日志一直 `Connecting.....____` 或报 `Timed out waiting for packet header`：
    1. 关闭正在运行的串口监视器，它会占用串口。

2. 把 `upload_speed` 改为 `460800` 或 `115200` 再试。
3. 如果板子上有 BOOT/IO0 按键：按住它，再点上传，看到 `Writing at…` 后松开。很多 HoloCubic 主板靠自动下载电路，没有按键；这时换线、换口，并检查焊接。 - 官方 Q&A 原话：用 AIO 上位机刷机卡住不动，请检查主板（不要怀疑工具或固件）。

3.5 关于「首次完整刷机」和官方上位机 AIO_Tool

- PlatformIO 的 Upload 会把 **bootloader + 分区表 + boot_app0 + 固件** 一起写入，所以一般不需要另外处理 bootloader。这是 PlatformIO/Arduino-ESP32 的通常行为【未实机验证】。
- 官方另有「上位机」`AIO_Tool`（仓库 `AIO_Tool/`，Python 源码，作者主要提供 Windows `CubicAIO_Tool.exe`）。它需要 `bootloader`、`partitions.bin`、`boot_app0.bin` 和固件 bin 四个文件。仓库里 `AIO_Tool/base_bin/` 带的是 `bootloader_qio_80m.bin`，README 写的是 `bootloader_dio_40m.bin`，两者不一致【已核对文档】。**Mac 用户建议直接用 PlatformIO**，不必用上位机。
- 系统设置（Wi-Fi 等）存在板载 Flash 的文件系统中（`/sys.cfg` 等，SPIFFS）。如果以前刷的固件分区布局不同，刷完后可能需要重新配 Wi-Fi【推断，未实机验证】。

3.6 串口监视，确认开机正常

打开 Serial Monitor（115200），按一下板子复位或重新插线，正常会看到：

- `Initialization MPU6050 now, Please don't move.`【已核对代码】：此时**把小电视平放、别动**，它在做陀螺仪自动校准。默认开启自动校准（`auto_calibration_mpu = 1`），每次开机执行。
- 如果没插 SD 卡，会有挂载失败的提示，**不影响开机**。README 原文：无论是否插 TF 卡、MPU6050 是否正常、是否连 Wi-Fi，都不影响系统启动和显示。

---

4. 连接 Wi-Fi（使用原固件自带的 WebServer）

【已核对代码】要点：

- 热点名 `HoloCubic_AIO`，**无密码**（`network.cpp: AP_SSID`，`open_ap(..., password=NULL)`）。
- 热点 IP 固定为 **`192.168.4.2`**（`network.cpp: local_ip(192,168,4,2)`）。
- 配置后台也支持域名 `http://holocubic`，但部分浏览器不支持，推荐用 IP。
- Wi-Fi 名称/密码保存在 Flash（`/sys.cfg`），**与 SD 卡无关**。

4.1 手势基础（所有 APP 通用）

【已核对代码 + README】：

手势	在菜单里	在 APP 里
左倾 / 右倾	切换到下一个 / 上一个 APP	由 APP 自己决定
向前倾并保持约 1 秒	**进入** APP	—
向后仰并保持约 1 秒	—	**退出** APP（返回菜单）

固件需要连续 3 次检测到同一方向（长按）才判定为进入/退出，所以**要保持住**，不是点一下。

如果你的方向感觉是反的，可以在 Web 设置里调整「操作方向（0~15可选）」(`mpu_order`)。

4.2 配网步骤

1. 小电视开机后，左/右倾找到 **WebServer**（菜单里名字是 `WebServer`），前倾 1 秒进入。
2. 屏幕显示 `WebServer Start`、`Domain: holocubic`，稍等后显示两个 IP：**Local IP**（连上路由器后的地址）和 **AP IP**（`192.168.4.2`）。
3. Mac 右上角 Wi-Fi → 选择 **`HoloCubic_AIO`**（无密码）。macOS 可能提示「无互联网连接」，这是正常的。
4. 浏览器打开 **`http://192.168.4.2`**，进入「系统设置」页（`/sys_setting`）。
5. 填写：- `WiFi SSID_0(2.4G)`：你家 Wi-Fi 名称，大小写要一致。- `WiFi Passwd_0`：Wi-Fi 密码。- 可选：「开机自启的APP名字」填 **`AIStatus`**，以后开机直接进入 AI 状态界面。
6. 点保存。
7. 后仰 1 秒退出 WebServer。Mac 切回你家的 Wi-Fi。

> 页面上的字段文字以实际网页为准。代码里 SSID 字段名为 `ssid_0`、自启字段为 `auto_start_app`【已核对代码】。

---

5. 进入 AIStatus，查看 IP

1. 回到菜单，**第一个 APP** 就是 **AIStatus**（图标：白色胶囊 + 三个圆点），前倾 1 秒进入。
2. 屏幕底部先显示 `WiFi connecting...`，连上后变成 `192.168.x.x:8080`。串口同时打印：`[AIStatus] HTTP server: http://192.168.x.x:8080/status`
3. 屏幕布局（240×240）：- 顶部：agent 名字。- 中部：彩色圆点 + 状态大字，下面是消息文本（最多约 3 行，过长加 `...`）。- 下方：当前状态持续时间，以及其他 agent 的小列表（最多 4 个）。
4. 退出：后仰 1 秒。退出后 RGB 灯会恢复成你原来的灯效设置。

5.1 强烈建议：固定 IP

每次重启 IP 都可能变，变了脚本就发不到。做法：

1. 浏览器登录路由器后台（常见地址 `192.168.1.1` 或 `192.168.0.1`，背面贴纸上有）。
2. 找「DHCP 静态分配 / IP 与 MAC 绑定 / 地址保留」。
3. 在设备列表里找到小电视（主机名是 `HoloCubic`【已核对代码 network.cpp】，或看 MAC），绑定当前 IP。

各品牌路由器菜单名不同【未验证具体品牌】。

5.2 中文显示：把字体拷到 SD 卡（推荐）

AIStatus 内置的英文字体不含中文。要显示中文和常用 emoji，需要把字体文件放到 microSD 卡上。

1. 用读卡器把 microSD 卡插到 Mac，确认格式是 **FAT32**（「磁盘工具」里显示「MS-DOS (FAT32)」）。如果需要重新格式化：选「MS-DOS (FAT)」+「主引导记录」，**会清空卡**。

2. 把压缩包里的 `放置到内存卡/font/ai_cn16.bin` 拷到 SD 卡上，**路径必须是**：`SD卡根目录/font/ai_cn16.bin` 固件读取的路径是 `S:/font/ai_cn16.bin`，其中 `S:` 是工程里 LVGL 文件系统驱动的盘符【已核对代码 `lv_fs_fatfs.c`】。
3. 在 Finder 里推出 SD 卡，插回小电视，重启，再进入 AIStatus。
4. 串口日志出现下面这行，就说明加载成功；agent 名称、消息文本、小列表会改用中文字体显示：`[AIStatus] CN font loaded: 4042 glyphs, RAM 24808 B, free heap ... B` 如果出现 `font S:/font/ai_cn16.bin not found, use built-in font`，说明没找到文件，会**自动回退**到内置英文字体。APP 照常可用，只是中文显示为空白或方块。

字体说明【已核对代码 / 已编译】：

- 内容：ASCII、GB2312 一级常用汉字 3755 个、中文标点、常用符号（← → ✓ ● ★ …）、约 80 个黑白 emoji（✅❌⚠🔥💡🤔🎉🚀👍 等），共 4042 个字形。
- 规格：16px，4bpp 抗锯齿，文件约 744 KB。由 `lv_font_conv` 用 Noto Sans SC（Medium）+ Noto Emoji 生成。
- **不会把整个字体读进内存**。ESP32（pico32，没有 PSRAM）放不下 700 多 KB，LVGL 自带的 `lv_font_load` 会整个加载，所以没有用它。改为流式读取：内存里只放码点表（约 16 KB）和 48 个字形的缓存（约 8.6 KB），共约 25 KB；其余按需从 SD 卡读取。
- 空闲内存不足（需要的 25 KB 再加 40 KB 余量）时会放弃加载并回退，避免 WiFi 异常。
- 状态大字（thinking/done 等）始终用内置英文字体。
- 退出 APP 时释放字体占用的内存。
- 电脑端脚本会把字库里没有的 emoji 替换成 `[庆祝]`、`[测试]` 这类文字或直接删掉，并按 UTF-8 字符边界截断到 120 字节。固件端也按字符边界截断，不会切出半个汉字。
- 不在一级字库里的生僻字会显示为空白。

如需重新生成字体：见 `AIO_Firmware_PIO/tools/build_ai_font.sh`。

---

6. 电脑端接入

以下命令都在「终端」中执行。`ai_status_bridge` 换成你实际解压的位置，例如 `~/Downloads/holocubic_ai_status/ai_status_bridge`。

6.1 安装通知脚本

```
mkdir -p ~/.holocube
cp ai_status_bridge/holocube_notify.sh ai_status_bridge/sanitize_text.py ai_status_bridge/
ai_cn16_charset.txt \
   ai_status_bridge/test_states.sh ai_status_bridge/grok_wrapper.sh ~/.holocube/
chmod +x ~/.holocube/*.sh
echo "192.168.1.50" > ~/.holocube_ip       # ← 换成你屏幕上看到的 IP（不写端口默认 8080）
```

也可以用环境变量临时指定：`export CUBE_IP=192.168.1.50`，它的优先级高于文件。

脚本特点【已在 Linux 用模拟服务测试】：

- 在后台发送，连接超时 1 秒、总超时 2 秒。
- 小电视关机也**不会卡住 AI 工具**，脚本总是正常退出。

6.2 用 curl 测试（确认网络通）

```
curl http://192.168.1.50:8080/status
```

应返回类似 `{"ok":true,"app":"AIStatus","ip":"...","agents":[]}`。然后：

```
curl -X POST http://192.168.1.50:8080/status \
  -H 'Content-Type: application/json' \
  -d '{"agent":"claude","state":"thinking","text":"hello cube"}'
```

屏幕应显示 `claude / thinking / hello cube`，圆点和灯变蓝。

6.3 跑一遍所有状态

```
~/.holocube/test_states.sh
```

每 2 秒切换一次 idle→thinking→working→waiting→done→error，最后加入 codex、grok 两条，用来检查多 agent 列表。

6.4 Claude Code

事件名已对照 Claude Code 官方 hooks 文档（code.claude.com/docs/en/hooks）核对。

1. 打开（不存在就新建）`~/.claude/settings.json`：`bash mkdir -p ~/.claude && open -a TextEdit ~/.claude/settings.json` TextEdit 要用「格式 → 制作纯文本」。用 VS Code 打开更方便：`code ~/.claude/settings.json`。
2. **文件是空的**：把 `ai_status_bridge/claude_settings_snippet.json` 的全部内容粘进去。 **文件已有内容**：只把其中 `"hooks": {...}` 这一段合并到最外层的大括号里，注意逗号。已经有 `hooks` 的，把各事件数组合并。
3. 映射关系：

Claude 事件	何时触发	小电视状态
`UserPromptSubmit`	你发出提问	thinking
`PreToolUse`	调用工具前（读写文件、跑命令）	working
`Notification`	需要你确认权限 / 闲置等待输入	waiting
`Stop`	回答结束	done
`StopFailure`	因 API 错误结束	error

1. 每个 hook 都设置了 `"async": true`（后台运行，不拖慢 Claude）。
2. 在 Claude Code 中输入 `/hooks`，能看到这些 hook 就说明加载成功。

6.5 Codex CLI

对照 OpenAI Codex 官方配置文档核对：

- `notify` 只能写在**用户级** `~/.codex/config.toml`，写在项目里的 `.codex/config.toml` 会被忽略。

- 目前只有 `agent-turn-complete`（一轮结束）这一个事件，JSON 作为最后一个参数传给程序。

```
whoami          # 记下你的用户名
mkdir -p ~/.codex && code ~/.codex/config.toml
```

在文件**最上方**（任何 `[xxx]` 小节之前）加一行，把 `你的用户名` 替换掉：

```
notify = ["/Users/你的用户名/.holocube/holocube_notify.sh", "--codex"]
```

- 效果：Codex 每答完一轮，小电视显示 `codex / done`，消息为最后一句回复。
- 局限：`notify` **没有**「等待确认」事件，所以 Codex 不会显示 waiting/working。Codex 新版本另有生命周期 hooks 机制，本指南未配置【未验证】。

6.6 Grok

根据 docs.x.ai 文档【未实机验证】：

- Grok CLI（`grok`）支持 hooks，个人配置放在 `~/.grok/hooks/*.json`，格式与 Claude 相同。
- 它**默认也读取 `~/.claude/settings.json` 里的 Claude hooks**，所以完成 6.4 后，Grok 很可能已经自动生效。
- 脚本检测到环境变量 `GROK_HOOK_EVENT` 时，会自动把名字显示为 `grok`。
- 检查是否加载：在 Grok 里输入 `/hooks`，或在终端运行 `grok inspect`。
- 如果想单独配置，把 `claude_settings_snippet.json` 复制为 `~/.grok/hooks/holocube.json`。注意不要和 Claude 那份同时生效，否则会重复上报。
- 不用 hooks 的办法：用包装脚本启动，开始显示 working，退出显示 done 或 error： `bash ~/.holocube/grok_wrapper.sh grok` 这个办法适用于任何命令行工具。

---

7. 问题排查

7.1 方块端（硬件 / 固件）

现象	可能原因	解决
`ls /dev/cu.*` 没有新设备	充电线不能传数据；缺驱动；USB 扩展坞问题	换数据线；直插 Mac；按 1.3 装 CP210x/CH340 驱动并在「隐私与安全性」允许，然后重启
上传报 `Timed out waiting for packet header` / 卡在 `Connecting...`	串口被监视器占用；自动下载电路或焊接问题；速率太高	关监视器；`upload_speed` 降到 115200；有 BOOT 键就按住再上传；检查主板焊接（官方 Q&A：刷机卡住先查主板）
`Resource busy` / `could not open port`	串口被其他程序占用	关闭 Serial Monitor、其他串口工具、Arduino IDE
反复重启，尤其在连 Wi-Fi 时；发热严重	供电不足；屏幕板/主板 LDO 方案发热、压降大	换供电充足的 USB 口或充电头；参考仓库 硬件版本说明.md：Metal 屏幕板 v1.2+ 加了一路 LDO 分担电流，v2.0/v2.1 可改焊 DC-DC 方案（此时去掉主板 LDO）以根治发热

现象	可能原因	解决
白屏、花屏、黑屏但串口正常	屏幕焊接/排线问题；屏幕型号不符（需 1.3 寸 ST7789 240×240）；`TFT_eSPI` 配置被改	检查屏幕焊接；不要随意修改 `lib/TFT_eSPI/User_Setup.h`；背光引脚在 `src/common.h` 中为 IO5（PEAK 版为 IO12）
串口出现 `sdcard_mount(): f_mount failed: (3) The physical drive cannot work`	没插卡、卡不兼容、卡槽焊接问题、格式不对	没有 SD 卡时 AIStatus 仍可用（只是没有中文）；其他 APP 需要时：换官方推荐的 32G 卡并格式化为 FAT32，检查卡槽焊接（SD 用 SPI：SCK14/MISO26/MOSI13/SS15）。代码里 SD 的 SPI 时钟设为 80 MHz，个别卡不稳定属已知兼容性问题（官方 issue 有此报错）
AIStatus 一直显示 `WiFi connecting...`	没配 Wi-Fi；用的是 5G；密码错；信号弱	按第 4 章配网；确认 2.4G；SSID 大小写一致；看串口日志；靠近路由器
Mac 上 curl 超时、访问不了 IP	不在同一网络；访客网络 / AP 隔离；IP 变了；没进入 AIStatus；端口写错	Mac 和小电视连同一个 Wi-Fi，且不是访客网络；关闭路由器「AP 隔离 / 客户端隔离」；看屏幕底部最新 IP；**必须在 AIStatus APP 内**；端口是 **8080**（80 是 WebServer APP 的）
手势不灵、乱切换、进不去 APP	开机自动校准时被移动；MPU6050 焊接/I2C（SDA32/SCL33）问题；方向设置不对	开机时平放不动等校准完成；倾斜要**保持约 1 秒**；Web 设置里调整「操作方向（0~15可选）」；仍不行检查 MPU6050 焊接（官方 Q&A：IIC 焊接有问题会导致切换不了页面）
中文显示为空白/方块	SD 卡上没有字体文件、路径不对，或 SD 卡没挂载，已回退到英文字体	按 5.2 把 `ai_cn16.bin` 放到 SD卡根目录`/font/`；看串口是否有 `CN font loaded`；SD 卡用 FAT32
串口提示 `font needs ... free heap ... -> skip`	空闲内存不足，主动放弃加载中文字体	关掉其他占内存的功能后重启，再直接进入 AIStatus（可设为开机自启）
个别汉字显示空白	该字不在 GB2312 一级字库（3755 字）里	属正常限制；需要的话扩充 `tools/sc.txt` 重新生成字体
emoji 变成 [庆祝] 之类的文字或消失	字库只含约 80 个常用 emoji，其余由电脑端脚本替换或删除	属设计行为；可在 `sanitize_text.py` 里修改映射
第一次显示一段中文时略有停顿	字形需要从 SD 卡读取，之后会缓存	正常现象【未实机测速】
编译报 `arduino.h: No such file` / `EMOJI.h`	原仓库头文件大小写不一致，区分大小写的文件系统上会报错	本补丁已修正；macOS 默认不区分大小写，一般不会遇到
编译报 Flash 空间不足（`region flash overflowed`）	开了太多 APP	在 `src/app/app_conf.h` 中把不用的 APP 宏改成 0
第一次编译特别慢或下载失败	首次需要下载 espressif32 平台和工具链	保持网络通畅，重试；必要时开代理
刷新固件后 Wi-Fi 设置丢了	分区布局变化导致 Flash 文件系统被重建	按第 4 章重新配网

7.2 电脑端

现象	原因	解决
手动 curl 正常，AI 工具触发后屏幕没变化	IP 文件没写对；脚本没有执行权限；hooks 没加载	`cat ~/.holocube_ip`；`chmod +x ~/.holocube/*.sh`；Claude 里输入 `/hooks` 检查
想单独测试脚本	—	`CUBE_IP=你的IP ~/.holocube/holocube_notify.sh working claude "test"`
`settings.json` 改完 Claude 报配置错误	JSON 格式错（逗号、括号）	把内容粘到在线 JSON 校验器检查，或运行 `python3 -m json.tool ~/.claude/settings.json`
Codex 没反应	`notify` 写在了某个 `[小节]` 下面；路径里用了 `~`；写在了项目配置里	放到文件最顶部；用 `/Users/用户名/...` 绝对路径；写在 `~/.codex/config.toml`
同一事件显示两次 / 名字变来变去	Claude 与 Grok 共用一份 hooks，或 Grok 同时加载了两份	只保留一份配置

---

8. 附录

8.1 状态对照表

state	含义	屏幕/灯颜色
`idle`	空闲	灰 `#808080`
`thinking`	思考中	蓝 `#3D7BFF`
`working`	执行工具/写代码	橙 `#FFB000`
`waiting`	等你确认/输入	粉 `#FF40C0`
`done`	完成	绿 `#20D060`
`error`	出错	红 `#FF3030`

8.2 API 说明（端口 8080，仅在 AIStatus APP 内可用）

POST `/status`，`Content-Type: application/json`

```
{"agent": "claude", "state": "working", "text": "Editing main.cpp"}
```

字段	说明
`agent`	名字，最长 23 字节（UTF-8 安全截断），不区分大小写匹配；缺省为 `agent`
`state`	必须是上表 6 个值之一（大小写不敏感），否则返回 400
`text`	可选，超过 127 字节会按 UTF-8 字符边界截断并加 `...`；中文需要 SD 卡字体（见 5.2）

- 同时最多记录 4 个 agent，满了会替换最久没更新的那个。
- 最近一次更新的 agent 显示为主画面。
- 状态没变时，计时不清零。

成功返回 200，内容与 GET 相同。

GET `/status`（健康检查），返回示例：

```
{"ok":true,"app":"AIStatus","ip":"192.168.1.50","uptime_s":1234,
 "agents":[{"agent":"claude","state":"done","text":"...","elapsed_s":12}]}
```

8.3 常用命令速查

```
ls /dev/cu.*                                          # 找串口
pio run -e HoloCubic_AIO_Releases -t upload           # 编译+上传
pio device monitor -b 115200                          # 串口日志
curl http://$(cat ~/.holocube_ip):8080/status         # 健康检查
~/.holocube/holocube_notify.sh done claude "hi"       # 手动发送
~/.holocube/test_states.sh                            # 全状态测试
```

8.4 相关代码位置

内容	路径
AIStatus APP	`AIO_Firmware_PIO/src/app/ai_status/`
APP 开关	`AIO_Firmware_PIO/src/app/app_conf.h`
APP 安装顺序	`AIO_Firmware_PIO/src/HoloCubic_AIO.cpp`
热点名 / AP IP	`AIO_Firmware_PIO/src/network.cpp`
网页配置	`AIO_Firmware_PIO/src/app/server/`
引脚定义	`AIO_Firmware_PIO/src/common.h`
手势识别	`AIO_Firmware_PIO/src/driver/imu.cpp`

8.5 尚未验证的事项（如实列出）

1. AIStatus 固件和 SD 中文字体**未在真机上运行**（包括流式字体渲染、SD 读取速度、实际空闲内存）：界面排版、RGB 灯颜色、与 WebServer APP 来回切换后端口是否正常释放，都需要实测。
2. Mac 上的驱动安装、PlatformIO 上传流程未在 Mac 实机验证。
3. Grok 读取 Claude hooks 只根据官方文档，未实测。Codex 新版 hooks 未配置。
4. 路由器 DHCP 绑定的具体菜单名因品牌而异。
5. 换固件后是否需要重新配网，取决于之前刷的版本。